Software Test Plan

Railway Ticket Reservation System

Version 2.0 | Revised 12 November 2026 | Team 15 | UE24CS341A

This revision specifies the delivered C++ application and its verification evidence. It aligns the submission documents with the implementation and identifies the remaining team acceptance actions. It is prepared for review; signatures and acceptance are not implied.

Team member	USN	Role
Deeptanshu Kumar	PES1UG24AM357	Developer
Amar	PES1UG24AM340	Tester
Aaryaprasad	PES1UG24AM331	Product Owner

Revision history

Version	Date	Change
1.0	2 October 2026	Original submitted baseline; retained in repository history
2.0	12 November 2026	Reconcile actual stack, interfaces, policies, traceability and current verification evidence

Application baseline: 6b5464298c949806ebba0b7bd1d903c962b5255d. The final feature integration is main commit fb1b4d55957ab8022e136396d656bec870c08375, merged by Amar in PR 12 on 3 November 2026. Current evidence was generated on 12 November 2026.

1 Introduction

This plan defines verification of the Railway Ticket Reservation System against its FR, NFR and security requirements. It covers scope, strategy, environment, roles, schedule, test cases, traceability and reporting. Version 2 reconciles actual C++ test tooling and new evidence; planned test steps are not represented as personal manual execution or signed acceptance.

- Course Mini Project Deliverables Parts 1, 2 and 3; evaluation checklist; final demo and viva FAQs.
- Repository README, CMakeLists.txt, src/api, src/services, src/db and tests at the application baseline above.
- IEEE 830 style requirements structure; IEEE 1016 style software design description; IEEE 829 style test planning structure, adapted to the supplied course templates.
- docs/SUBMISSION_INDEX.md for the deliverable inventory; docs/DEVIATIONS.md for baseline differences; docs/reports for actual results.

2 Test items

- Passenger account/profile/session services; train search, schedule, fares and inventory.
- Passenger validation, booking preview/confirmation, PNR, tickets and history.
- Cancellation policy, refund recording, seat restoration and booking ownership.
- Administrator train lifecycle, audit/access logging, validation, TLS, concurrency, recovery and responsive UI.

3 Features to be tested

Requirement	Module	Test case references
RTS-F-001	UserService / Routes	TC-REG-01, TC-REG-02
RTS-F-002	AuthService / Routes	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04

Requirement	Module	Test case references
RTS-F-003	TrainService / Routes	TC-SEARCH-01, TC-SEARCH-03, TC-SEARCH-04
RTS-F-004	TrainService / web/app.js	TC-SEARCH-02
RTS-F-005	TrainService / seat_reservations	TC-AVAIL-01, TC-AVAIL-02
RTS-F-006	BookingService / web/app.js	TC-BOOK-01
RTS-F-007	BookingService / Validation	TC-BOOK-02, TC-BOOK-06
RTS-F-008	BookingService / Database	TC-BOOK-03
RTS-F-009	BookingService / bookings	TC-BOOK-04
RTS-F-010	BookingService / Database	TC-BOOK-05, TC-CONC-01
RTS-F-011	BookingService / web/app.js	TC-TICKET-01
RTS-F-012	BookingService / Routes	TC-HISTORY-01
RTS-F-013	BookingService	TC-CANCEL-01, TC-CANCEL-04
RTS-F-014	BookingService	TC-CANCEL-02
RTS-F-015	BookingService / Database	TC-CANCEL-03, TC-CANCEL-05
RTS-F-016	Authorization / BookingService	TC-SEC-03
RTS-F-017	AdminTrainService / AdminRoutes	TC-ADMIN-01, TC-ADMIN-02, TC-ADMIN-03, TC-ADMIN-04
RTS-F-018	AuditService / audit_events	TC-AUDIT-01, TC-AUDIT-03
RTS-NF-001	TrainService / BookingService / browser	TC-PERF-01
RTS-NF-002	Database / BookingService	TC-CONC-01, TC-CONC-02
RTS-NF-003	Routes / Validation	TC-ERR-01, TC-ERR-02
RTS-NF-004	Database / executable	TC-REC-01, TC-CANCEL-05
RTS-NF-005	PasswordHasher / UserService	TC-SEC-01
RTS-NF-006	Authorization / AuthService	TC-SEC-02, TC-ADMIN-02
RTS-NF-007	web/index.html / style.css / app.js	TC-UX-01
RTS-NF-008	AuditService / Database	TC-AUDIT-01, TC-AUDIT-03, TC-REC-01
RTS-SR-001	PasswordHasher / AuthService	TC-SEC-01, TC-SEC-07
RTS-SR-002	AuthService / Authorization	TC-AUTH-03, TC-SEC-02, TC-SEC-03
RTS-SR-003	TlsConfig / main / cpp-httplib	TC-SEC-04
RTS-SR-004	Validation / Database / web/app.js	TC-SEC-05, TC-BOOK-02, TC-ERR-02
RTS-SR-005	Log / Routes / web/app.js	TC-SEC-06, TC-AUDIT-02

Named automated tests are listed in the Test Plan and source/test_cases.json. Passing unit/API checks, system assertions and selected browser observations are distinct evidence types. Human UAT and PO/mentor acceptance remain pending; the matrix is not a fabricated sign-off.

4 Features not tested

- Real railway operational control, live railway feeds and physical railway hardware.
- Payment-provider settlement, bank refunds and external financial-system behavior.
- Production-scale distributed load, power-loss durability, full penetration-test certification and every browser/device/accessibility combination. These exclusions do not remove the required local NFR/security checks.

5 Test approach and strategy

5.1 Security validation

Control	Procedure and evidence
Password storage	Inspect DB salted PBKDF2 hashes; negative credentials; rehash; TC-SEC-01 and automated XML
Authorization	Missing/revoked tokens, ordinary-user admin requests, cross-owner booking access; TC-SEC-02/03

Control	Procedure and evidence
Transport	Verify TLS configuration/headers plus actual executable handshake with hostname verification and temporary trusted test certificate; TC-SEC-04
Input safety	Malformed JSON, wrong types, SQL/script-like values, boundaries and a 64 KB limit; TC-SEC-05
Sensitive logging	Inspect redaction, no auth body/header/query logging and safe errors; TC-SEC-06

5.2 Levels and techniques

GoogleTest exercises pure validators, hashing, service rules, database rollback and HTTP API integration. The stdlib Python runner launches the actual executable with fresh temporary SQLite files and ephemeral ports. Browser observations exercise actual DOM forms and rendering. Apply equivalence partitioning, boundary values, state transitions, negative authorization and concurrent inventory tests. Human UAT remains a separate tester/product-owner activity.

5.3 Entry criteria

- A buildable identified revision with initialized controlled data and reachable APIs.
- Isolated synthetic passenger/admin credentials and no use of production reservation data.
- Required compiler, CMake, OpenSSL, Python, GoogleTest and reporting tools available; planned scenarios not blocked by a known build failure.

5.4 Exit criteria and acceptance

- Execute all planned cases or document unexecuted/blocked cases and obtain genuine disposition.
- No open critical defect blocks core workflows; mandatory FR/NFR/security evidence is reviewed.
- Store Unit, System Validation, Code Coverage and Branch Coverage reports under docs; keep actual run dates, revision and raw results.
- Tester and PO complete human acceptance and reviewer approvals; automated checks alone do not satisfy signature or demo requirements.

5.5 Data isolation and reset

GoogleTest uses controlled in-memory fixtures with reduced PBKDF2 iterations where appropriate. The executable runner uses the default server security configuration, random temporary passwords, disposable file-backed SQLite and temporary TLS certificates. It removes its own temporary data and logs no secrets. Browser rehearsal uses a separate synthetic database. Restart and compare data before reset; never delete a user database as a shortcut.

6 Test environment

Item	Executed configuration or procedure
Application	C++17/cpp-httplib/SQLite; Debug build with RTRS_COVERAGE=ON
Compiler/platform	GCC 16.2.1 on Fedora Linux; exact platform in execution-metadata.json
Unit/tooling	GoogleTest/CTest, gcov/gcovr 8.8; src-only coverage excluding main.cpp
System runner	Python stdlib, localhost HTTP/HTTPS, openssl executable, temporary DB
Browser	Chrome isolated context; desktop 1366×900, mobile 390×844
Seed route	Pune Jn→Mumbai CSMT; Deccan Express 11007; SL fare 190, fresh capacity 300
Policy	Default 24-hour cutoff, 90 percent refund, whole-rupee rounding; UTC departure
Scope limit	Small local dataset; not production latency or physical-device certification

7 Schedule and responsibilities

Milestone	Course planned window	Actual evidence
Requirements/design baseline	28 Sept 2026	Original SRS/SAD/STP revisions retained in Git
Backlog preparation	12 Oct 2026	Historical Jira snapshot 13 Oct; GitHub migration 12 Nov
Sprint 1	19–23 Oct 2026	Search/booking delivery 20 Oct; earlier auth foundation
Sprint 2	26–30 Oct 2026	Cancellation delivery 29 Oct
Optional Sprint 3 / testing	2–6 Nov 2026	Admin/security final feature delivery 3 Nov
Submission verification refresh	12 November 2026	Fresh 201-test, 122-check, coverage and browser reports

Person	Responsibility
Deeptanshu Kumar	Developer; USN PES1UG24AM357
Amar	Tester; USN PES1UG24AM340
Aaryaprasad	Product Owner; USN PES1UG24AM331

Deeptanshu provides build, implementation and automated verification. Amar reviews the plan, executes human acceptance, tracks defects and reviews integration. Aaryaprasad confirms requirements/priorities/policy and acceptance. Actual individual repository evidence is separated in docs/demo/VIVA_PREP.md.

8 Test deliverables

- This Test Plan with detailed cases and RTM; source/test_cases.json and source/requirements.json.
- reports/Unit_Test_Report.pdf and .md; unit-tests.xml; ctest-results.xml.
- reports/System_Validation_Report.pdf and .md; system_validation.json.
- reports/Code_Coverage_Report.pdf and .md; reports/Branch_Coverage_Report.pdf and .md; coverage JSON and HTML.
- reports/Browser_Validation_Report.md and genuine UI screenshots.
- Defect dispositions, actual human acceptance and videos remain team actions; blank approvals are not manufactured.

9 Risks and mitigation

Risk	Mitigation
Dirty/stale data	Fresh fixtures; identified source/binary; per-run raw records
Double booking/partial write	Concurrent buyers, DB trigger and rollback failure scenarios
Stale availability	Preview versus confirmation conflict tests
Timing/cutoff flakiness	Future journey fixture and explicit UTC/default policy
Overstated coverage	Separate line/branch, exclude main/browser explicitly; retain raw counts
Historical misattribution	Preserve original Git/Jira capture times; label retrospective course planning

10 Assumptions and dependencies

Seed records are synthetic. APIs and routes reflect the actual revision. Policy defaults are used for reproducible tests but require product acceptance. A successful test does not establish an unexecuted browser scenario or a human reviewer identity. The original Jira snapshot is dated 13 October; later live updates need reconnection.

11 Suspension and resumption

Suspend testing if the executable will not start, critical test data is unavailable, or a blocker prevents the core flow. Record the blocking check and do not report an incomplete runner as PASS. Resume after fixing the blocker, rebuilding and repeating affected checks. Preserve failed-run evidence and correct the RTM.

12 Requirement traceability and execution interpretation

Requirement	Module	Test case references
RTS-F-001	UserService / Routes	TC-REG-01, TC-REG-02
RTS-F-002	AuthService / Routes	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04
RTS-F-003	TrainService / Routes	TC-SEARCH-01, TC-SEARCH-03, TC-SEARCH-04
RTS-F-004	TrainService / web/app.js	TC-SEARCH-02
RTS-F-005	TrainService / seat_reservations	TC-AVAIL-01, TC-AVAIL-02
RTS-F-006	BookingService / web/app.js	TC-BOOK-01
RTS-F-007	BookingService / Validation	TC-BOOK-02, TC-BOOK-06
RTS-F-008	BookingService / Database	TC-BOOK-03
RTS-F-009	BookingService / bookings	TC-BOOK-04
RTS-F-010	BookingService / Database	TC-BOOK-05, TC-CONC-01
RTS-F-011	BookingService / web/app.js	TC-TICKET-01
RTS-F-012	BookingService / Routes	TC-HISTORY-01
RTS-F-013	BookingService	TC-CANCEL-01, TC-CANCEL-04
RTS-F-014	BookingService	TC-CANCEL-02
RTS-F-015	BookingService / Database	TC-CANCEL-03, TC-CANCEL-05
RTS-F-016	Authorization / BookingService	TC-SEC-03
RTS-F-017	AdminTrainService / AdminRoutes	TC-ADMIN-01, TC-ADMIN-02, TC-ADMIN-03, TC-ADMIN-04
RTS-F-018	AuditService / audit_events	TC-AUDIT-01, TC-AUDIT-03
RTS-NF-001	TrainService / BookingService / browser	TC-PERF-01
RTS-NF-002	Database / BookingService	TC-CONC-01, TC-CONC-02
RTS-NF-003	Routes / Validation	TC-ERR-01, TC-ERR-02
RTS-NF-004	Database / executable	TC-REC-01, TC-CANCEL-05
RTS-NF-005	PasswordHasher / UserService	TC-SEC-01
RTS-NF-006	Authorization / AuthService	TC-SEC-02, TC-ADMIN-02
RTS-NF-007	web/index.html / style.css / app.js	TC-UX-01
RTS-NF-008	AuditService / Database	TC-AUDIT-01, TC-AUDIT-03, TC-REC-01
RTS-SR-001	PasswordHasher / AuthService	TC-SEC-01, TC-SEC-07
RTS-SR-002	AuthService / Authorization	TC-AUTH-03, TC-SEC-02, TC-SEC-03
RTS-SR-003	TlsConfig / main / cpp-httplib	TC-SEC-04
RTS-SR-004	Validation / Database / web/app.js	TC-SEC-05, TC-BOOK-02, TC-ERR-02
RTS-SR-005	Log / Routes / web/app.js	TC-SEC-06, TC-AUDIT-02

Named automated tests are listed in the Test Plan and source/test_cases.json. Passing unit/API checks, system assertions and selected browser observations are distinct evidence types. Human UAT and PO/mentor acceptance remain pending; the matrix is not a fabricated sign-off.

The 201 GoogleTests are not 201 detailed plan cases. The 46 cases below map to named tests and/or system/browser evidence. A passing automated check supports its asserted behavior; it does not claim every written manual step was personally replayed. System JSON contains 122 actual assertions. TC-REC-01 and TC-UX-01 use their specific reports. TC-PERF-01 combines local search/booking measurements with the stated limits.

13 Defect and change management

Record an actual failed scenario with revision, inputs, expected and actual results, severity, owner and evidence. Retest after a fix and link the commit/PR. No synthetic defect closure list or invented review is supplied. Changes to requirements, architecture or policy update the SRS, SAD, Test Plan and affected cases together.

14 Detailed test cases

Unless noted otherwise, use a fresh isolated database, synthetic users and a future date at least three days ahead. Apply authentication required by the case. Steps define reproducible tests; current outcomes are available in the linked reports rather than implied human signatures.

TC-REG-01 Create passenger account

Requirements: RTS-F-001. Method: Automated / manual replay.

Inputs and preconditions: New email; name Test Rider; phone 9876543210; password Journey123.

1. Reset isolated DB.

2. POST all four registration fields.

3. Inspect response and users row.

Expected: 201; normalized unique email; PASSENGER role; no password/hash in response.

Automated evidence: RegisterApi.CreatesAccountAndNeverEchoesSecrets.

TC-REG-02 Reject duplicate and invalid registration

Requirements: RTS-F-001. Method: Automated / manual replay.

Inputs and preconditions: Existing email with changed case; missing phone; malformed JSON.

1. Register baseline account.

2. Repeat email using uppercase.

3. Submit missing phone then malformed JSON.

4. Compare account row count.

Expected: 409 duplicate; 400 invalid; no extra accounts; field errors identify invalid data.

Automated evidence: RegisterApi.DuplicateEmailReturns409; RegisterApi.ValidationErrorsListEachField; RegisterApi.MalformedAndNonObjectJsonReturn400.

TC-AUTH-01 Valid login

Requirements: RTS-F-002. Method: Automated / manual replay.

Inputs and preconditions: Registered passenger and correct password.

1. POST /api/auth/login.

2. Use returned token as Authorization: Bearer on GET /api/users/me.

3. Inspect sessions storage.

Expected: 200 login/profile; opaque token is usable; stored token_hash differs from raw bearer.

Automated evidence: LoginApi.SuccessReturnsTokenAndUserWithoutHash; SecurityApi.SessionTokensAreStoredOnlyAsHashes.

TC-AUTH-02 Invalid login and lockout

Requirements: RTS-F-002. Method: Automated / manual replay.

Inputs and preconditions: Unknown email; wrong password; five failures for one account.

1. Compare wrong-password and unknown-user responses.

2. Repeat failures five times.

3. Try correct password during lock.

4. Advance injected clock past 300 seconds and retry.

Expected: Generic 401 responses match; lock returns 429; after expiry valid login succeeds.

Automated evidence: LoginApi.BadCredentialsReturnTheSame401Body; LoginApi.RepeatedFailuresReturn429; AuthServiceTest.LockExpiresAndSuccessResetsCounter.

TC-AUTH-03 Reject invalid session

Requirements: RTS-F-002, RTS-SR-002. Method: Automated / manual replay.

Inputs and preconditions: Missing, garbage and expired tokens.

1. Request protected booking/profile endpoints with each token.

2. Advance test clock beyond 24-hour TTL.

3. Retry expired token.

Expected: 401; no protected state disclosed or changed.

Automated evidence: AuthServiceTest.SessionExpires; BookingCancelSecurityApi.QuoteAndCancellationRejectMissingInvalidAndRevokedSessions.

TC-AUTH-04 Logout and password session revocation

Requirements: RTS-F-002, RTS-SR-002. Method: Automated / manual replay.

Inputs and preconditions: Two valid sessions for same user.

1. Logout one session and request profile.

2. Change password using current password from remaining session.

3. Verify other sessions revoked and current session retained.

Expected: Logged-out token fails; password update keeps current session and revokes others.

Automated evidence: LogoutApi.LogsOutAndTokenStopsWorking; ProfileApiTest.PasswordChangeKeepsCurrentSessionAndRevokesOthers.

TC-SEARCH-01 Search ordered route

Requirements: RTS-F-003. Method: Automated / manual replay.

Inputs and preconditions: Fixture Alpha → Beta; future valid date D.

1. GET /api/trains/search?source=Alpha&destination=Beta&date=D.

2. Compare returned train and route against fixture.

3. Reverse route.

Expected: 200 matches forward stop order; reverse route cannot match that forward-only schedule.

Automated evidence: SearchApi.ReturnsMatchingTrainsAsJson.

TC-SEARCH-02 Inspect schedule and fares

Requirements: RTS-F-004. Method: Automated / manual replay.

Inputs and preconditions: Known train ID with ordered stops and class fares.

1. Search fixture route.

2. GET train schedule and fares.

3. Compare number/name, stops, times, duration and fares.

Expected: Summary and details match fixture; stop order/day offsets and class fare values preserved.

Automated evidence: ScheduleApi.ReturnsTrainAndStops; FaresApi.ReturnsClassFares.

TC-SEARCH-03 No results and invalid criteria

Requirements: RTS-F-003. Method: Automated / manual replay.

Inputs and preconditions: Unknown route; absent date; invalid calendar date.

1. Search unmatched valid route.

2. Omit required parameters.

3. Submit invalid date.

Expected: 200 with empty list for no matches; 400 with field errors for invalid criteria.

Automated evidence: SearchApi.NoMatchReturnsEmptyList;
SearchApi.MissingParametersReturnFieldErrors.

TC-SEARCH-04 Filter and sort

Requirements: RTS-F-003. Method: Automated / manual replay.

Inputs and preconditions: Multiple trains with different class, fare and departure values.

1. Search using class, maxFare and departure range.

2. Sort by price/duration/departure with order.

3. Submit invalid filter.

Expected: Only eligible trains returned in requested order; invalid filter yields field error.

Automated evidence: FilterApi.AppliesClassFareAndSortParameters;
FilterApi.InvalidFilterReturnsFieldError.

TC-AVAIL-01 Class inventory equation

Requirements: RTS-F-005. Method: Automated / manual replay.

Inputs and preconditions: Train class capacity C; reserve n seats on date D.

1. GET availability before booking.

2. Book n passengers.

3. GET availability for D and D+1.

4. Compare reservation sums.

Expected: Available seats are C−n on D and unaffected on D+1.

Automated evidence: AvailabilityApi.ReturnsClassWiseAvailability;
BookingAvailabilityApi.AvailabilityIsPerDate.

TC-AVAIL-02 Sold-out and bad availability query

Requirements: RTS-F-005. Method: Automated / manual replay.

Inputs and preconditions: Class with zero remaining capacity; invalid date; unknown ID.

1. Fill class.

2. Inspect availability and attempt preview.

3. Request invalid date and unknown train.

Expected: Zero available; sold-out preview rejected; invalid date 400 and missing train 404.

Automated evidence: BookingAvailabilityApi.SoldOutClassIsRejected; AvailabilityApi.MissingOrInvalidDateReturns400; AvailabilityApi.UnknownTrainReturns404.

TC-BOOK-01 Preview without mutation

Requirements: RTS-F-006. Method: Automated / manual replay.

Inputs and preconditions: Logged-in passenger; valid train/date/class; two passengers.

1. Count bookings/reservations.

2. POST /api/bookings/preview.

3. Inspect total fare, count and availability.

4. Recount rows.

Expected: 200; totalFare=2×farePerPassenger; no booking or reservation created.

Automated evidence: BookingPreviewApi.ReturnsFareAndAvailability.

TC-BOOK-02 Passenger field boundaries

Requirements: RTS-F-007, RTS-SR-004. Method: Automated / manual replay.

Inputs and preconditions: Names with digits/script, age 0/121/non-numeric, invalid gender.

1. POST preview and confirm with invalid data.

2. Inspect indexed field errors.

3. Query booking/passenger rows.

Expected: 400; correct passenger index and field; no partial booking.

Automated evidence: BookingValidationApi.RejectsInvalidNameAgeAndGender; BookingValidationApi.ReportsTheCorrectPassengerIndex; InputValidationApi.OverlongAndScriptPassengerNamesAreRejected.

TC-BOOK-03 Stale preview conflict

Requirements: RTS-F-008. Method: Automated / manual replay.

Inputs and preconditions: One remaining seat; two logged-in passengers.

1. Preview one seat for user A.

2. User B confirms last seat.

3. A confirms original preview.

4. Inspect all state.

Expected: B succeeds; A gets 409; no A booking/passenger/reservation/success audit.

Automated evidence: BookingConfirmApi.RejectsWhenSeatsRanOutAfterPreview.

TC-BOOK-04 Distinct persisted PNR

Requirements: RTS-F-009. Method: Automated / manual replay.

Inputs and preconditions: Capacity for multiple bookings.

1. Confirm two bookings.

2. Check PNR format/uniqueness.

3. Query bookings table and retrieve tickets by PNR.

Expected: Each PNR is a distinct ten-digit string stored on its booking.

Automated evidence: BookingPnrApi.ReturnsTenDigitPnr; BookingPnrApi.PnrIsUniqueAcrossBookings; BookingPnrApi.PnrIsStoredOnTheBooking.

TC-BOOK-05 Seat decrement exactly once

Requirements: RTS-F-010. Method: Automated / manual replay.

Inputs and preconditions: Capacity 10; three valid passengers.

1. Read starting availability.

2. Confirm three passengers once.

3. Inspect passengers and linked reservation.

4. Read availability.

Expected: 201; three distinct seats; one quantity-3 linked reservation; availability drops from 10 to 7.

Automated evidence: BookingConfirmApi.ReservesSeatsAndReducesAvailability; BookingConfirmApi.CreatesConfirmedBookingWithSeatsAndPnr.

TC-BOOK-06 Passenger count boundaries

Requirements: RTS-F-007. Method: Automated / manual replay.

Inputs and preconditions: Passenger arrays of size 0, 1, 6 and 7.

1. Preview each array on class with sufficient seats.

2. Confirm valid boundaries in isolated fixtures.

Expected: 0 and 7 rejected; 1 and 6 accepted if all fields valid and capacity sufficient.

Automated evidence: BookingPreviewApi.RejectsMissingPassengers; BookingPreviewApi.RejectsMoreThanSixPassengers; BookingAvailabilityApi.AllowsBookingExactlyTheRemainingSeats.

TC-TICKET-01 Owner ticket by PNR

Requirements: RTS-F-011. Method: Automated / manual replay.

Inputs and preconditions: Confirmed booking; owner bearer token.

1. GET /api/bookings/{pnr}.

2. Compare PNR, date, class, train, passengers, seats and fare with confirmation.

3. Request unknown PNR.

Expected: 200 correct ticket; unknown PNR 404; URL uses PNR, not numeric internal ID.

Automated evidence: BookingTicketApi.OwnerCanViewTicket; BookingTicketApi.UnknownPnrReturnsNotFound.

TC-HISTORY-01 Isolated newest-first history

Requirements: RTS-F-012. Method: Automated / manual replay.

Inputs and preconditions: Two accounts; multiple confirmed/cancelled bookings.

1. GET history for new account.

2. Create bookings for both users and cancel one own booking.

3. GET history for each.

Expected: New account list empty; each user sees only own records newest-first including cancelled ticket details.

Automated evidence: BookingHistoryApi.ReturnsEmptyListWhenNoBookings; BookingHistoryApi.ReturnsBookingsNewestFirst; BookingHistoryApi.DoesNotIncludeOtherUsersBookings.

TC-CANCEL-01 Eligible owner cancellation

Requirements: RTS-F-013. Method: Automated / manual replay.

Inputs and preconditions: Own confirmed future booking outside 24-hour cutoff.

1. GET cancellation quote using numeric booking id.

2. POST or PUT /api/bookings/{id}/cancel.

3. Retrieve ticket.

Expected: Quote eligible; cancellation 200; ticket CANCELLED with cancelledAt and recorded refund.

Automated evidence: BookingCancelApi.CancelsEligibleBooking.

TC-CANCEL-02 Refund quote and floor rule

Requirements: RTS-F-014. Method: Automated / manual replay.

Inputs and preconditions: Booking totalFare; default 90 percent and configured alternate percentage.

1. GET quote and verify no row mutation.

2. Compute floor(totalFare×percent/100).

3. Cancel and compare persisted refund.

Expected: Quote/persisted refund equal integer floor; configured percentage honored; no payment operation occurs.

Automated evidence: BookingRefundApi.QuotePreviewsRefundWithoutChangingBooking; BookingRefundApi.RefundUsesConfiguredPercentageAndRoundsDown.

TC-CANCEL-03 Restore own seats once

Requirements: RTS-F-015. Method: Automated / manual replay.

Inputs and preconditions: Two bookings on same train/date/class.

1. Cancel first booking.

2. Read availability and reservations.

3. Attempt second cancellation.

4. Rebook freed seats.

Expected: Only first booking quantity restored; repeated cancellation 409; freed seats reusable without active duplicates.

Automated evidence: BookingRestoreApi.CancellationRestoresOnlyItsOwnSeatsExactlyOnce; BookingRestoreApi.RebookingReusesFreedSeatsWithoutDuplicatingActiveSeats.

TC-CANCEL-04 Reject late or repeated cancellation

Requirements: RTS-F-013. Method: Automated / manual replay.

Inputs and preconditions: Booking within cutoff; already cancelled booking; alternate cutoff.

1. Request quote and cancel for late booking.

2. Cancel eligible booking twice.

3. Repeat with configured cutoff fixture.

Expected: Late quote ineligible, zero refund; cancellation 409; repeated cancellation 409; policy honored.

Automated evidence: BookingCancelApi.RejectsCancellationAfterCutoff; BookingCancelApi.RejectsSecondCancellation; BookingCancelApi.HonoursConfiguredCutoff.

TC-CANCEL-05 Cancellation rollback

Requirements: RTS-F-015, RTS-NF-004. Method: Automated / manual replay.

Inputs and preconditions: Confirmed booking; injected inventory deletion failure.

1. Capture ticket/reservation/audit state.

2. Inject failure inside cancellation transaction.

3. Attempt cancellation.

4. Compare state.

Expected: No partial cancellation, refund, seat restoration or success audit survives failure.

Automated evidence: BookingRestoreApi.InventoryFailureRollsBackCancellationAndRefund.

TC-SEC-01 Salted password storage

Requirements: RTS-NF-005, RTS-SR-001. Method: Automated / manual replay.

Inputs and preconditions: Two accounts with identical valid password.

1. Register accounts.

2. Inspect password_hash fields.

3. Verify correct/wrong password; inspect API responses.

Expected: Only PBKDF2 hashes stored; distinct salts; correct verifies, wrong fails; no password/hash disclosure.

Automated evidence: SecurityApi.PasswordsAreStoredOnlyAsSaltedHashes; PasswordHasher.SamePasswordGetsDifferentSalts.

TC-SEC-02 Current role authorization

Requirements: RTS-NF-006, RTS-SR-002. Method: Automated / manual replay.

Inputs and preconditions: Passenger, admin and revoked sessions.

1. Attempt admin list/create/update/audit with passenger/anonymous tokens.

2. Change admin role in DB and retry existing token.

3. Try role=ADMIN in registration.

Expected: 401/403 as appropriate; changed role enforced immediately; registration remains PASSENGER.

Automated evidence: SecurityApi.AdminRoleIsRecheckedOnEveryRequest; AdminAuthApi.PublicRegistrationCannotChooseAdminRole; SecurityApi.PassengerCannotUseAdminOperations.

TC-SEC-03 Ownership despite spoofed fields

Requirements: RTS-F-016, RTS-SR-002. Method: Automated / manual replay.

Inputs and preconditions: A owns booking; B has valid session and knows id/PNR.

1. As B read ticket, quote and cancel A booking.

2. Supply A owner id in body.

3. Call service owner check under test.

Expected: 404; no disclosure or state change; service enforces actual owner inside transaction.

Automated evidence: BookingCancelSecurityApi.AnotherUserCannotPreviewOrCancelEvenWithSpoofedOwner; BookingCancelSecurityApi.ServiceEnforcesOwnerInsideCancellationTransaction; BookingTicketApi.OtherUsersCannotViewTicket.

TC-SEC-04 Live TLS and configuration rejection

Requirements: RTS-SR-003. Method: System runner + configuration tests.

Inputs and preconditions: Temporary local certificate/key and explicit client trust; standalone executable.

1. Start with RTRS_TLS_CERT and RTRS_TLS_KEY.

2. Connect using trusted SSLContext and record negotiated protocol.

3. Run authenticated HTTPS request.

4. Start with only one TLS path.

Expected: Trusted local TLS connection succeeds; selected system run negotiates TLS 1.3; incomplete configuration fails startup.

Automated evidence: TlsConfigTest.RequiresBothCertificateAndKey; TlsConfigTest.AcceptsReadableCertificateAndKey.

TC-SEC-05 Injection and unsafe input

Requirements: RTS-SR-004. Method: Automated / manual replay.

Inputs and preconditions: SQL/script strings; control characters; wrong JSON types.

1. Submit adversarial login/search/registration/passenger/admin data.

2. Verify schema and row counts.

3. Inspect browser rendering of dynamic content.

Expected: Controlled rejection; no SQL execution, unintended write or script execution; dynamic content escaped/text assigned.

Automated evidence: InputValidationApi.ScriptAndInjectionInputsAreRejected; InputValidationApi.WrongJsonTypesCannotBypassValidation; LoginApi.SqlInjectionInEmailDoesNotAuthenticate.

TC-SEC-06 Secrets absent from diagnostics

Requirements: RTS-SR-005. Method: Automated / manual replay.

Inputs and preconditions: Distinct password/token markers; requests with query/control characters.

1. Run login and protected API calls.

2. Capture logs and error bodies.

3. Search markers and inspect browser request URLs.

Expected: No raw password/token in logs, URLs or errors; access targets omit query values and control characters.

Automated evidence: LoggingUtil.RedactsAuthenticationSecrets; LoggingUtil.AccessLogOmitsCredentialsAndQueryValues; LoggingUtil.RequestTargetDropsQueryAndControlCharacters.

TC-SEC-07 Hash upgrade and invalid encoded values

Requirements: RTS-SR-001. Method: Automated / manual replay.

Inputs and preconditions: Weaker stored PBKDF2 hash; malformed and extreme iteration counts.

1. Login successfully against weaker valid hash.

2. Inspect upgraded hash.

3. Attempt malformed/extreme hash verification.

Expected: Successful login upgrades weak hash; invalid/extreme values rejected without unbounded work.

Automated evidence: SecurityApi.LoginUpgradesWeakerStoredHashes; PasswordHasherSecurity.RejectsImplausibleIterationCounts.

TC-ADMIN-01 Create and update complete train

Requirements: RTS-F-017. Method: Automated / manual replay.

Inputs and preconditions: Admin; new train number; ordered stops; classes/capacity/fares.

1. POST /api/admin/trains.

2. Search and inspect details.

3. PUT /api/admin/trains/{id} with revised schedule/fare.

4. Recheck search/details and old booking fare.

Expected: Valid changes visible; existing booking fare snapshot retained.

Automated evidence: AdminTrainCreateApi.CreatesCompleteTrainVisibleInPassengerSearchAndDetails; AdminTrainUpdateApi.UpdatesSearchScheduleAndFaresWithoutChangingExistingBookingFare.

TC-ADMIN-02 Deny unauthorized admin access

Requirements: RTS-F-017, RTS-NF-006. Method: Automated / manual replay.

Inputs and preconditions: No token, passenger token and revoked admin token.

1. Attempt admin reads and writes with each.

2. Compare database before/after.

Expected: Anonymous/revoked 401; passenger 403; no write.

Automated evidence: AdminTrainCreateApi.ReadAndWriteRequireAnActiveAdminSession; AdminAuthApi.RejectsUnauthenticatedPassengerAndRevokedSessions.

TC-ADMIN-03 Deactivate with existing bookings

Requirements: RTS-F-017. Method: Automated / manual replay.

Inputs and preconditions: Active train and confirmed booking; stale preview.

1. PUT /api/admin/trains/{id}/deactivate.

2. Search and attempt stale confirmation.

3. Read existing ticket/history and cancel eligible booking.

4. Restart.

Expected: Train hidden; new confirmation rejected; existing ticket/history/cancel work; inactive state persists.

Automated evidence: AdminTrainDeactivateApi.HidesServiceAndRejectsBookingsFromStalePreviews; AdminTrainDeactivateApi.ExistingTicketsHistoryAndCancellationStillWork; AdminTrainDeactivateApi.MigratesExistingTrainRecordsAndPreservesInactiveStateOnRestart.

TC-ADMIN-04 Atomic validation and update guards

Requirements: RTS-F-017. Method: Automated / manual replay.

Inputs and preconditions: Duplicate train number; invalid stops; referenced class; capacity below assigned seats.

1. Submit each invalid create/update.

2. Inject class-update database failure.

3. Compare train/stop/class state.

Expected: 400/409 as implemented; no partial write; referenced class removal/capacity shrink rejected.

Automated evidence: AdminTrainCreateApi.DuplicateTrainNumberReturnsConflict; AdminTrainUpdateApi.CannotRemoveReferencedClassOrShrinkBelowAssignedSeats; AdminTrainUpdateApi.FailedClassUpdateRollsBackTrainAndStopChanges.

TC-AUDIT-01 Timestamped transition audit

Requirements: RTS-F-018, RTS-NF-008. Method: Automated / manual replay.

Inputs and preconditions: One successful booking followed by cancellation.

1. Read audit baseline.

2. Confirm then cancel.

3. Query audit_events and admin audit API.

4. Correlate booking/user/train/PNR/timestamp.

Expected: Exactly one event for each successful transition; actor and identifiers match persisted ticket.

Automated evidence: AuditApi.RecordsTimestampedBookingAndCancellationEvents; AuditApi.AdministratorCanReadTheAuditTrail.

TC-AUDIT-02 Secret-safe audit and access log

Requirements: RTS-SR-005. Method: Automated / manual replay.

Inputs and preconditions: Booking/cancellation with distinct credential markers.

1. Read generated audit fields and captured access logs.

2. Search raw credential/token markers.

Expected: Audit has identifiers/timestamp/refund only; no credentials or bearer values.

Automated evidence: LoggingUtil.AccessLogOmitsCredentialsAndQueryValues; AuditApi.RecordsTimestampedBookingAndCancellationEvents.

TC-AUDIT-03 No false success audit

Requirements: RTS-F-018, RTS-NF-008. Method: Automated / manual replay.

Inputs and preconditions: Failed booking and injected cancellation transaction failure.

1. Capture audit count.

2. Trigger failed operations.

3. Compare audit and booking state.

Expected: No success event for rolled-back/failed operation.

Automated evidence: AuditApi.FailedBookingDoesNotLeaveAnAuditEvent; BookingRestoreApi.InventoryFailureRollsBackCancellationAndRefund.

TC-PERF-01 Separate search and booking p90

Requirements: RTS-NF-001. Method: System runner + browser evidence.

Inputs and preconditions: Standalone server; local fixture; four clients; 20 searches and 20 bookings.

1. Run system runner on isolated DB.

2. Record latency per request and status.

3. Compute separate p90 for search and booking and retain raw samples.

4. Inspect selected UI flow responsiveness separately.

Expected: Both p90 ≤3 seconds; no failed requests silently discarded; record machine/load; no production throughput inference.

Automated evidence: PerformanceApi.P90SearchLatencyStaysUnderThreeSeconds.

TC-CONC-01 Concurrent booking consistency

Requirements: RTS-NF-002, RTS-F-010. Method: Automated / manual replay.

Inputs and preconditions: Limited capacity; competing clients against same train/date/class.

1. Launch concurrent booking attempts.

2. Collect responses.

3. Query active seat assignments, reservation sums and capacity.

Expected: No duplicate active seat; no overselling; successful quantities reconcile with inventory.

Automated evidence: ConcurrencyApi.ConcurrentBookingsAllocateEachSeatOnlyOnce.

TC-CONC-02 Database duplicate-seat defense

Requirements: RTS-NF-002. Method: Automated / manual replay.

Inputs and preconditions: Two confirmed bookings with same inventory key.

1. Insert passenger seat on first booking.

2. Directly attempt same seat for second.

3. Inspect rows after rejection.

Expected: Trigger rejects duplicate active seat; existing allocation preserved.

Automated evidence: ConcurrencyDatabase.UniqueSeatTriggerRejectsDirectDuplicates.

TC-ERR-01 Safe validation and server errors

Requirements: RTS-NF-003. Method: Automated / manual replay.

Inputs and preconditions: Malformed JSON; unknown endpoint; injected database failure.

1. Submit malformed request.

2. Request unknown route/wrong method.

3. Trigger service failure.

4. Inspect response and state.

Expected: Controlled JSON errors; no database/stack/credential detail; no partial mutation.

Automated evidence: SafeErrorsApi.MalformedJsonIsRejectedSafely;
SafeErrorsApi.UnknownRouteReturnsGenericJson;
AdminTrainCreateApi.DatabaseFailureRollsBackTrainStopsAndClasses.

TC-ERR-02 Body and identifier boundaries

Requirements: RTS-NF-003, RTS-SR-004. Method: Automated / manual replay.

Inputs and preconditions: Body >64 KiB; oversized numeric ID; wrong JSON field types.

1. POST oversized body.

2. GET oversized resource path.

3. Submit wrong types.

4. Confirm server health.

Expected: Controlled rejection (413 oversized payload or safe endpoint validation); no crash and health remains available.

Automated evidence: SafeErrorsApi.OversizedRequestBodyIsRejectedSafely;
SafeErrorsApi.OversizedResourceIdDoesNotCrashTheServer;
InputValidationApi.WrongJsonTypesCannotBypassValidation.

TC-REC-01 Restart and abrupt-stop persistence

Requirements: RTS-NF-004, RTS-NF-008. Method: System runner.

Inputs and preconditions: File-backed isolated DB; committed booking/cancellation/audit state.

1. Start real executable, write committed state and stop cleanly.

2. Restart same database and compare state.

3. Write committed state, terminate abruptly, restart.

4. Run PRAGMA integrity_check and reconcile inventory/audit.

Expected: Committed records retained after both stops; integrity_check=ok; available capacity and active reservations reconcile.

Automated evidence: See System_Validation_Report and raw JSON.

TC-UX-01 Responsive passenger workflow

Requirements: RTS-NF-007. Method: Browser validation.

Inputs and preconditions: Mobile 390×844; desktop 1366×900; browser against real server.

1. At mobile, search, preview, confirm ticket, view and cancel; inspect overflow/reachability.

2. At desktop, register/login/search and inspect overflow.

3. Retain screenshots and viewport/action metadata.

4. Record unexercised desktop steps as not independently repeated.

Expected: Selected mobile booking/cancel and desktop registration/login/search are usable without horizontal page overflow; full desktop booking/cancel remains a separate planned repetition.

Automated evidence: See Browser_Validation_Report and screenshots.

15 Metrics and reporting

Current local verification: 201 tests across 49 suites pass; 122 system assertions pass; line coverage is 1476/1558 (94.7%); branch coverage is 3162/6778 (46.7%); function coverage is 167/167 (100%) in the measured scope. Search p90 is 0.002256 seconds and booking p90 is 0.006130 seconds, each with 20 requests and four clients. The course testing window is historical; these fresh runs are dated 12 November 2026.

Review planned case execution, pass/fail/blocked counts, requirement coverage, defect severity/retest state, latency method and coverage counts separately. No minimum coverage percentage was given in the five handouts. An absence of failed automated checks does not prove zero defects or completed human acceptance.

16 Review and approvals

Reviewer	Review responsibility	Signature and actual date
Amar	Test Plan and verification review	Pending
Aaryaprasad	Requirements and policy acceptance	Pending
Rajeshwari Adarakatti	Mentor evaluation	Pending